

SpringSecurity를 이용한 인증 서비스 구현

👉 로그인하기

Form 로그인 방식에서 UsernamePasswordAuthenticationFilter를 사용한다.

Form 로그인 방식에서는 클라이언트가 username과 password를 전송하면 Security 필터를 통과하는데 **UsernamePasswordAuthenticationFilter에서 회원 검증을 진행을 시작한다**. 회원 검증의 경우 UsernamePasswordAuthenticationFilter가 호출한 AuthenticationManager를 통해 진행하며 DB에서 조회한 데이터를 UserDetailsService를 통해 받는다. 우리의 SecurityConfig에서 **formLogin 방식을 disable 했기 때문에 기본적으로 활성화 되어 있는 해당 필터는 동작하지 않는다**.

```
http.formLogin((auth) -> auth.disable());
```

따라서,

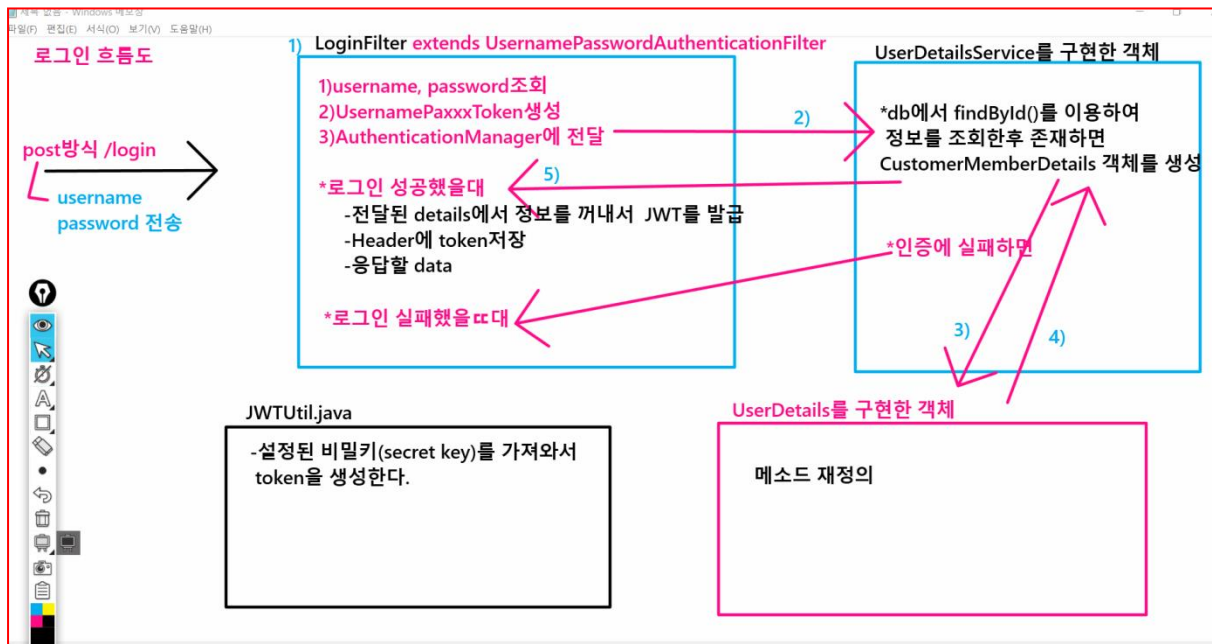
Form Login 을 진행하는 필터는 **UsernamePasswordAuthenticationFilter**이다
우리는 이 필터를 disable 시키고 Customizing 해서 JWT 방식 로그인을 진행한다.

Filter	Added by
CsrfFilter	HttpSecurity#csrf
UsernamePasswordAuthenticationFilter	HttpSecurity#formLogin
BasicAuthenticationFilter	HttpSecurity#httpBasic
AuthorizationFilter	HttpSecurity#authorizeHttpRequests

- **LoginFilter** extends UsernamePasswordAuthenticationFilter
- **LoginFilter**를 SecurityConfig에 등록해야 작동한다.
- <https://docs.spring.io/spring-security/reference/servlet/authentication/passwords/form.html#servlet-authentication-form>

👉 로그인 흐름도

<http://localhost:9000/login>으로 요청이 들어오면 LoginFilter가 가로챈다.



구현 해보기

Project 5—security05_Login

Group ID : web

Artifact ID :mvc

Dependency : Dev Tool, Spring Security, Spring Web, Lombok , Oracle Driver, JPA

1. Security04_Register프로젝트 복사해서 security05_Login 이름을 변경한다.

2. Dependency 추가 -jwt / gson 라이브러리

```
<!-- https://mvnrepository.com/artifact/io.jsonwebtoken/jjwt-api -->
<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-api</artifactId>
<version>0.12.3</version>
</dependency>
<!-- https://mvnrepository.com/artifact/io.jsonwebtoken/jjwt-impl -->
<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-impl</artifactId>
<version>0.12.3</version>
<scope>runtime</scope>
```

```

</dependency>
<!-- https://mvnrepository.com/artifact/io.jsonwebtoken/jjwt-jackson -->
<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-jackson</artifactId>
<version>0.12.3</version>
<scope>runtime</scope>
</dependency>
<!-- https://mvnrepository.com/artifact/com.google.code.gson/gson -->
<dependency>
<groupId>com.google.code.gson</groupId>
<artifactId>gson</artifactId>
<version>2.10.1</version>
</dependency>

```

3. properties파일에 security key 설정

```
spring.jwt.secret=SahL5G83mGzruJzffw-eD7RqphsXluJPBwxu0ncBFo
```

: secret 키 만들기

구글검색 <512 bit secret key generator

<https://www.awesometools.org/cryptography-tools/hmac-sha512-hash-generator/>

4. 작성 및 제공 파일

- 1) UserDetails 구현체 제공 – CustomMemberDetails
- 2) UserDetailsService 구현체 작성 – CustomDetailsService
- 3) JWTUtil(토큰생성) 제공
- 4) UsernamePasswordAuthenticationFilter 상속받은 LoginFilter 제공
- 5) JwtTokenTests 토큰 test파일 제공

5. Spring DOC 공식문서에서 필터확인

구글검색 < spring doc < <https://docs.spring.io/spring-framework/reference/index.html>

위 메뉴 중 Project 선택 < Spring Security < Get Started Servlet < Architecture < Security Filters

→<https://docs.spring.io/spring-security/reference/servlet/architecture.html#servlet-security-filters>

6. 코드 추가 – config/SecurityConfig.java

```
//AuthenticationManager 가 인자로 받을 AuthenticationConfiguraion 객체 생성자 주입
private final AuthenticationConfiguration authenticationConfiguration;

private final JWTUtil jwtUtil;

//AuthenticationManager Bean 등록
@Bean
public AuthenticationManager authenticationManager(AuthenticationConfiguration configuration) throws Exception {
    return configuration.getAuthenticationManager();
}
```

filterChain메소드안 추가부분

```
//필터 추가 LoginFilter()는 인자를 받음 (AuthenticationManager() 메소드에 authenticationConfiguration 객체를 넣어야 함)
//addFilterAt 은 UsernamePasswordAuthenticationFilter 의 자리에 LoginFilter 가 실행되도록 설정하는 것
http.addFilterAt(
    new LoginFilter( this.authenticationManager(authenticationConfiguration) , jwtUtil) ,
    UsernamePasswordAuthenticationFilter.class
);
```

```
56 //경로별 인가 작업
57 http.authorizeHttpRequests((auth) ->
58     auth
59         .requestMatchers("/index", "/members", "/members/**", "/boards").permitAll()
60         .requestMatchers("/admin").hasRole("ADMIN")
61         .anyRequest().authenticated());
62 //필터 추가 LoginFilter()는 인자를 받음 (AuthenticationManager() 메소드에 authenticationConfiguration 객체를 넣어
63 //addFilterAt은 UsernamePasswordAuthenticationFilter의 자리에 LoginFilter가 실행되도록 설정하는 것
64 http.addFilterAt(
65     new LoginFilter(
66         this.authenticationManager(authenticationConfiguration), //AuthenticationManager
67         jwtUtil), //JWTUtil
68     UsernamePasswordAuthenticationFilter.class);
69 return http.build();
70 }
71 }
```

7. 코드 추가 – security/CustomDetailsService.java

```

12  /**
13   * 인증처리 서비스 - repository에서 정보를 찾아서 인증...
14   * */
15   @Service
16   @Slf4j
17   @RequiredArgsConstructor
18   public class CustomDetailsService implements UserDetailsService {
19
20   private final MemberRepository memberRepository;
21
22   no usages
23   @Override
24   public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
25       log.info("username ={}", username); //아이디
26
27       // 디비에 id에 해당하는 정보가 있는지 찾기
28       Member findMember = memberRepository.findById(username);
29       if(findMember!=null){
30           log.info("findMember 찾았다~~ ={}", findMember);
31           return new CustomMemberDetails(findMember); // UserDetails
32       }
33       return null;
34   }

```

Token 발급 및 검증 단위 TEST

```

13  @SpringBootTest
14  @Slf4j
15  class JwtTokenTests {
16
17   @Autowired
18   private JWTUtil jwtUtil;
19
20
21   /**
22    * Registered claims : 미리 정의된 클레임
23    *   iss(issuer: 발행자),
24    *   exp(expiration time: 만료 시간),
25    *   sub(subject: 제목),
26    *   iat(issued At: 발행 시간),
27    *   jti(JWI ID: 토큰의 고유 식별자)
28    *
29    * Registered claims : 미리 정의된 클레임
30    * */
31   @Test
32   void tokenCreateTest() {
33       Member member = Member.builder().id("jang").memberNo(1L).role("ROLE_USER").name("희정").address("서울").build();
34
35       String token = jwtUtil.createJwt(member, role: "ROLE_USER", expiredMs: 1000L*60*10L); //10분
36       System.out.println("token = " + token);
37
38       System.out.println("---조회하기 (검증)---");
39       System.out.println("userName = " + jwtUtil.getUsername(token));
40       System.out.println("id = " + jwtUtil.getId(token));
41       System.out.println("Role = " + jwtUtil.getRole(token));
42       System.out.println("isExpired = " + jwtUtil.isExpired(token));
43   }
44 }

```

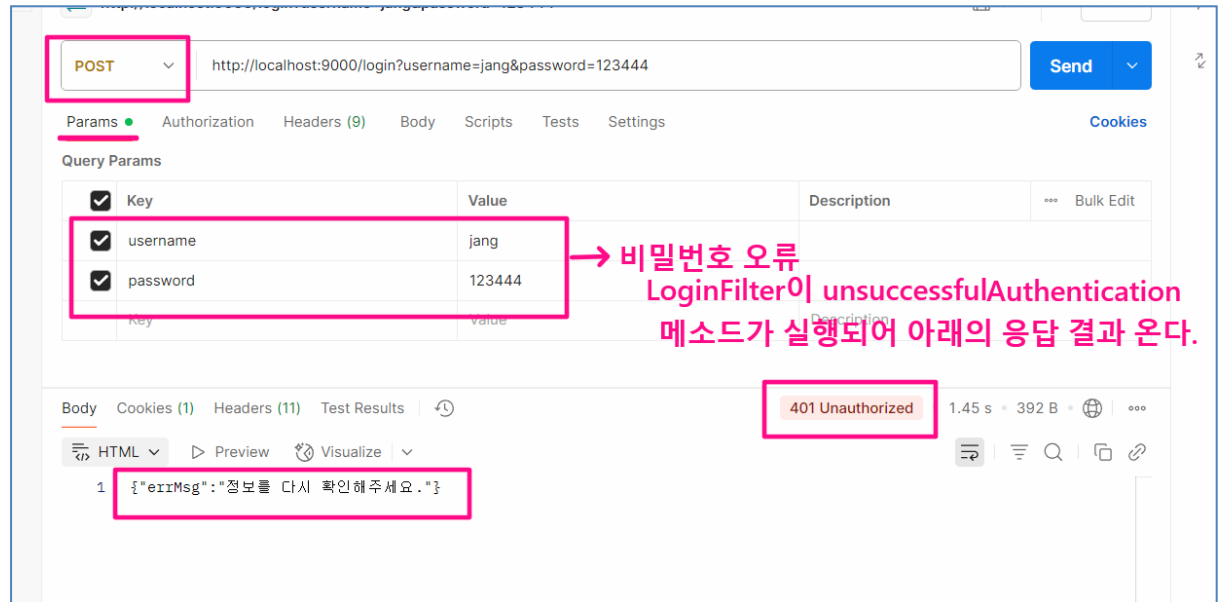
8. POST MAN으로 로그인 시도

MemberController에서 login이라는 요청이 없어도 스프링 시큐리티에서 로그인 요청 된다.

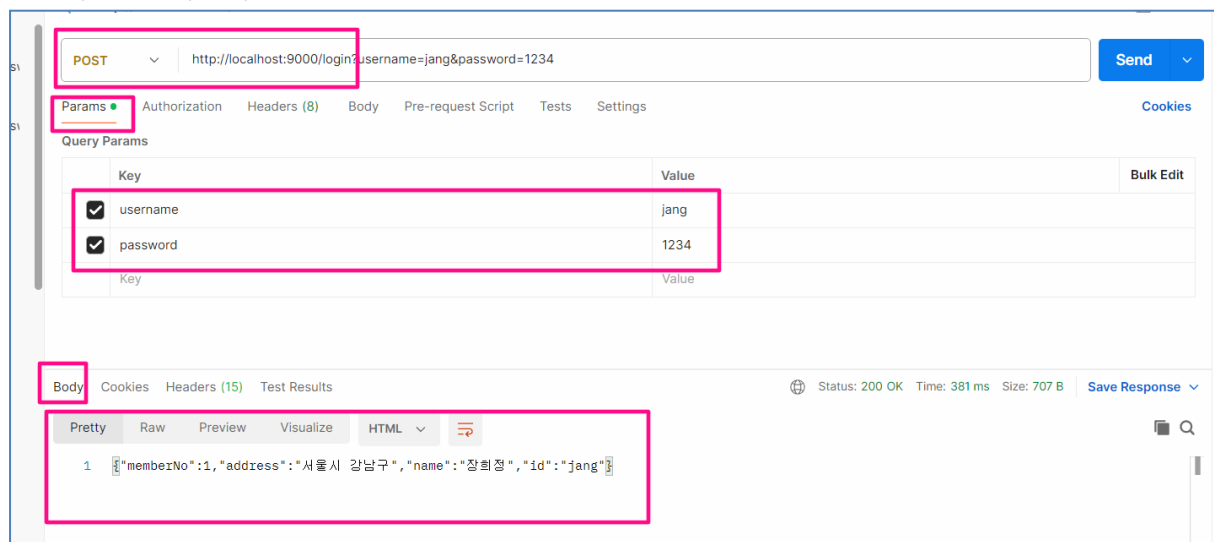
기본적으로 spring security 인증 요청 주소는 /login

사용자 아이디와 비밀번호는 username, password이다.

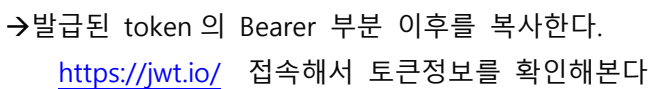
예) 로그인(인증) 실패



예) 로그인(인증) 성공



→ 발급된 Token 정보 확인하기



Encoded

PASTE A TOKEN HERE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpzZW50b3R5IiwiaWF0IjoxNjI3MzQzLCJleHAiOjE3MjYxMjc5NDN9.pZBG0xf84oz-rvbRUMN1_m_nXqF14ktRcxdcNapfyg
```

.을 통해서 헤더,페이로드, 시그니처 부분을 구분한다.

header, payload는 외부에서 디코딩해서 얼마든지 확인가능하다.
노출되어도 상관없는 데이터만 담아야 한다.
그래서 사용자의 비번은 담으면 안된다.

jwt는 발급처를 확인하는 용도라 이해하면 된다.
이 서버에서 인증한 클라이언트가 맞다...는 정도이다.

Decoded

EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE


```
{
  "alg": "HS256"
}
```

→ 암호 알고리즘 명시

PAYLOAD: DATA


```
{
  "username": "홍길동",
  "id": "kosta",
  "role": "ROLE_USER",
  "iat": 1726127343,
  "exp": 1726127943
}
```

→ 실제 사용자가 입력한 정보
(노출되어도 상관없는 정보만 담아야 한다)
jwt는 서버에서 발급된 것임을 인증할 수 있는 수단으로서 활용된다.

VERIFY SIGNATURE

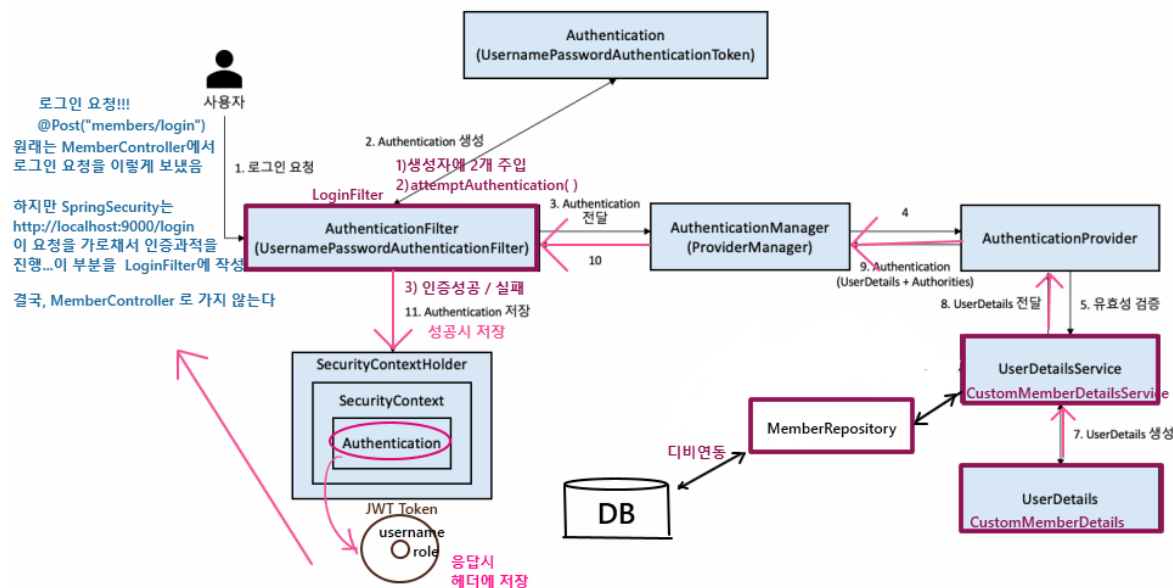
```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  your-256-bit-secret
)
```

☐ secret base64 encoded

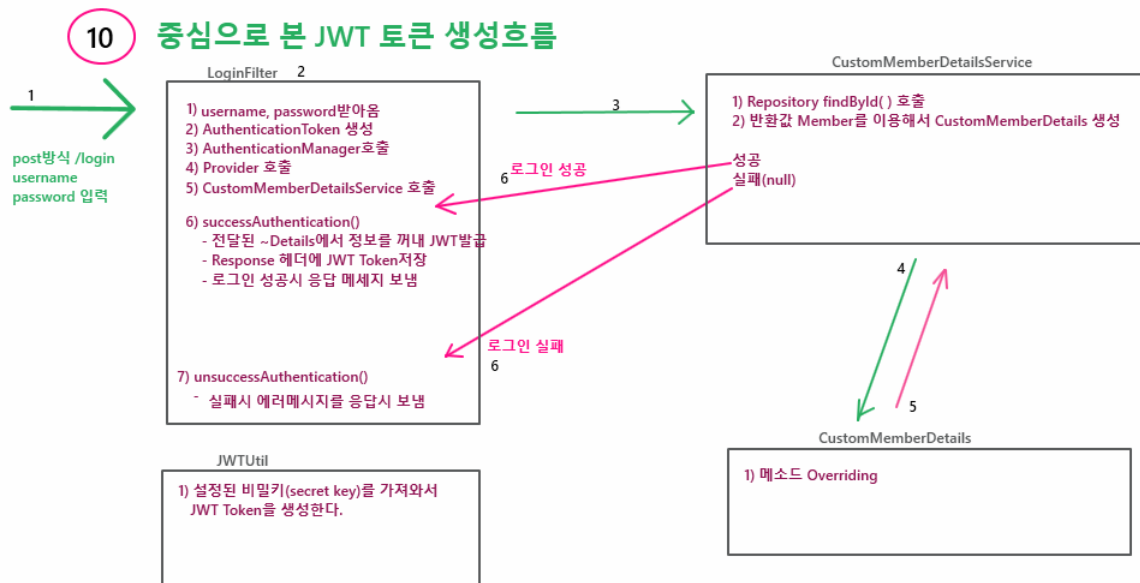
secret key를 제공한 서버에서만 검증을 진행할 수 있도록 하기 위함
base64방식으로 header,payload를 인코딩하고 최종적으로 secret key를 가지고 암호화 진행해서 나온 부분.

9. Spring Security 로그인 인증과정 흐름도

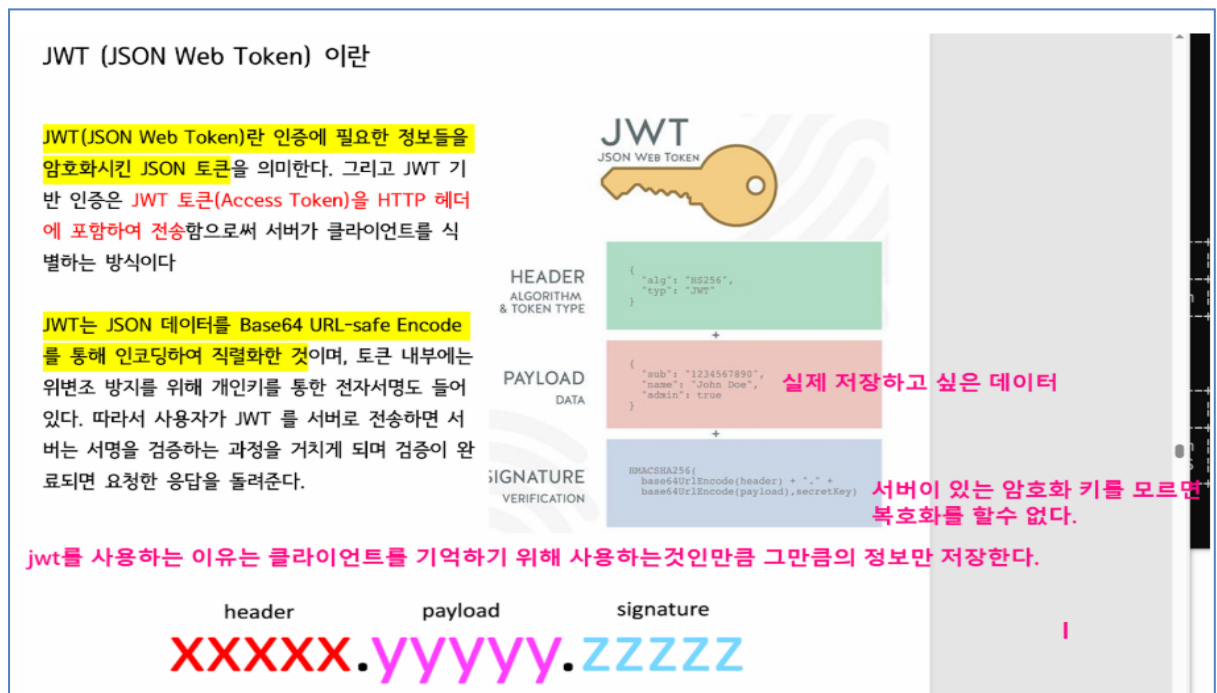
1)



2)



10. JWT 구성



JWT는 .을 기준으로 3부분으로 나뉜다.

Header.payload.signature

1)Header

헤더에서는 알고리즘과 토큰의 타입으로 구성되어 3번째 signature를 만드는데 사용되는 알고리즘이 들어간다.

HS256은 HMAC SHA 256을 의미하는데 이는 해시 알고리즘 중 한 가지이다.

타입의 값 JWT는 이 토큰을 JWT로 지정한다는 의미이다.

2)Payload

```
{
  "sub": "1234567890",
  "name": "장희정",
  "iat": 1516239022,
  "address": "오리역"
}
```

iss(issuer: 발행자),
 exp(expiration time: 만료 시간),
 sub(subject: 제목),
 iat(issued At: 발행 시간),
 jti(JWI ID: 토큰의 고유 식별자)

토큰을 누가 누구에게 발급했는지, 언제까지 유효한지, 사용자의 닉네임, 서비스상의 레벨, 관리자 여부등등 사용자에게 대한 정보를 저장하는 것을 클레임(Claim)이라고 한다.

데이터 각각의 Key를 claim이라 한다.

claim은 사용자가 원하는 Key와 Value로 구성된다.

단순 Base64 인코딩이 된 파트이며, 이는 누구나 디코딩하여 데이터 열람이 가능하기 때문에 password 같은 결정적인 요소들은 Payload에 담으면 안된다.

3) Signature

HMACSHA256(

base64UrlEncode(header) + "." +

base64UrlEncode(payload),

secret

)

서명 파트는 위와 같이 Base64 인코딩된 **Header** "." **Payload** 그리고 **secret** 이 필요하다.

secret 은 말 그대로 서버가 지정하는 비밀 코드를 말한다.

첫번째 Header 와 두번째 payload 그리고 서버에 감춰놓은 비밀값(secret_key)등 이 셋을 암호화 알고리즘에 넣고 실행하면 세 번째 서명값이 나온다.

만약, 해커가 token을 탈취하여 만료시간이나 정보를 변경한다 하여도 서버에 저장된 비밀 키를 알 수 없기 때문에 정보를 변경할 수 없다.

세 번째 서명 값이 일치하고 유효기간도 지나지 않았다면 그 사용자는 로그인된 회원으로서 인가(특정한 리소스에 접근할 수 있는 권한)를 받는다.

서버는 사용자의 상태를 따로 저장할 필요 없고 서버에 비밀키만 갖고 있으면 되기 때문 훨씬가

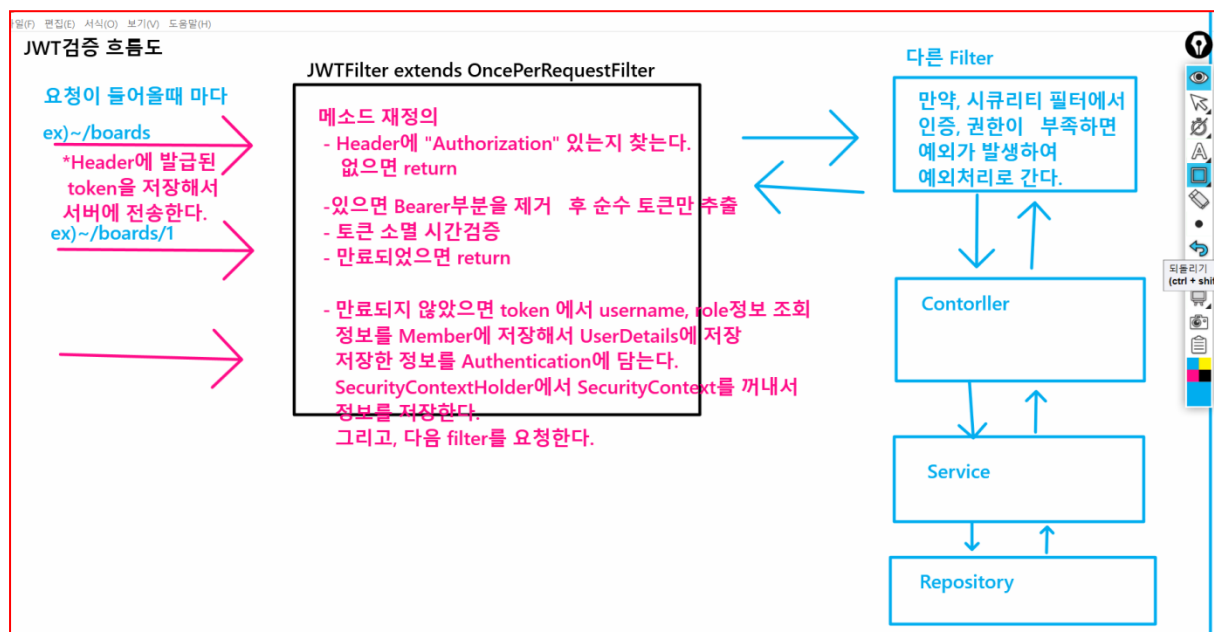
별다

이처럼 시간에 따른 흐름에 따라 어떤 상태값을 갖지 않는 것을 stateless라고 하며 Session은 반대로 stateful이라고 한다.

👉 JWT 검증하기

스프링 SecurityFilterChain 요청에 담긴 JWT 를 검증하기 위한 커스텀필터를 등록해야 한다. 해당 필터를 통해 요청 헤더 **Authorization** 키에 **JWT** 가 존재하는 경우 **JWT** 를 검증하고 **강제로 SecurityContextHolder** 에 세션을 생성한다. 이 세션은 **STATLESS** 상태로 관리되기 때문에 해당 요청이 끝나면 소멸 된다.

사용자 요청이 들어오면 HttpRequest 요청에 대해 딱 한번 만 실행하는 **OncePerRequestFilter** 를 상속한다. OncePerRequestFilter 는 **Spring Security** 에서 제공하는 추상 클래스 중 하나로, HTTP 요청을 처리하는 필터 중에서 **매 요청마다 한 번만 실행되도록 보장**하는 필터이다. 이 필터는 주로 필터 체인 내에서 다른 필터들이 여러 번 실행되는 것을 방지하고, 특정 작업을 요청 당 한 번만 수행하도록 할 때 사용된다.



👉 실습하기

Project 6- security6_LoginJWT

Group ID : web

Artifact ID : mvc

Dependency : Dev Tool, Spring Security, Spring Web, Lombok, mySQL , JPA

1. Security05_Login프로젝트를 복사해서 security06_LoginJWT 변경한다.

2. JWTFilter 제공

토큰 생성 후 다음 번 요청부터 서버에 제공되는 요청이 유효한지를 검증하기 위한 기능

3. Board관련 코드 작성

1) domain ,dto, repository, exception, service, controller –제공

2) SecurityConfig에 JWTUtil등록하기 -

```
//JWTFilter 등록
http.addFilterBefore(new JWTFilter(jwtUtil), LoginFilter.class);
```

4. Post Man 으로 요청 확인

1)전체게시물조회 : get 방식 <http://localhost:9000/boards> - permitAll()

2)등록 : post 방식 <http://localhost:9000/boards> - authenticated()

3)조회 : get 방식 <http://localhost:9000/boards/{id}> - authenticated()

4)수정 : put 방식 <http://localhost:9000/boards/{id}> - authenticated()

5)삭제 : delete 방식 <http://localhost:9000/boards/{id}> - authenticated()

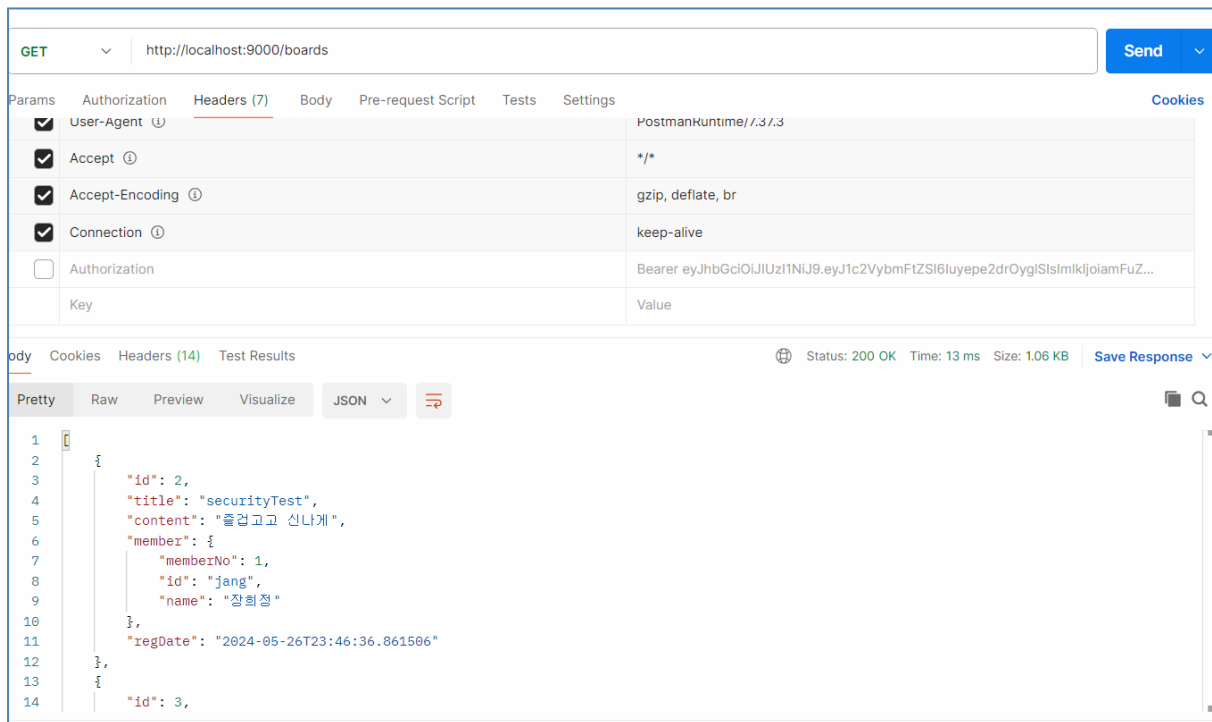
6)관리자 : post 방식 <http://localhost:9000/admin>

authenticated() + ROLE_ADMIN

전체게시물 조회

get 방식 <http://localhost:9000/boards> - permitAll()

: 인증여부 상관없이



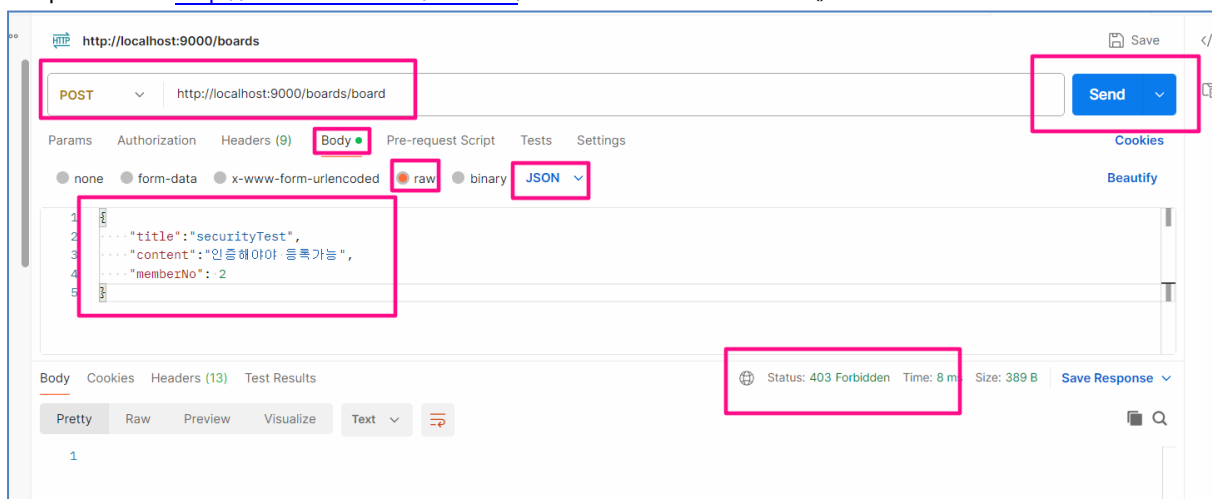
→콘솔창 확인

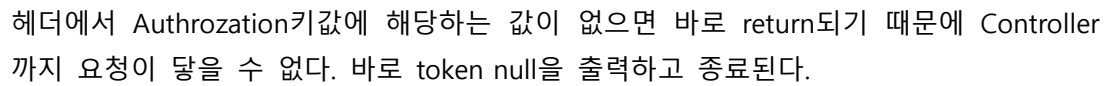
```

: authentication = AnonymousAuthenticationToken [Principal=anonymousUser, Credentials=[PROTECTED]]
: Authentication getName = anonymousUser
: Authentication authentication.getPrincipal() = anonymousUser
ole,b1_0.reg_date,b1_0.title,b1_0.update_date from board b1_0 join member m1_0 on m1_0.member_no=
  
```

등록 - 인증한 경우만 가능

post 방식 <http://localhost:9000/boards/board> - authenticated()

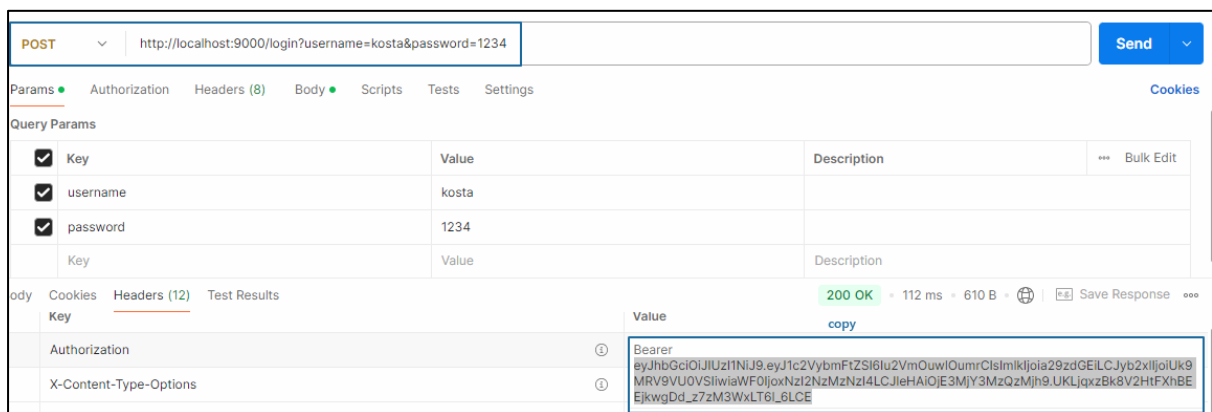
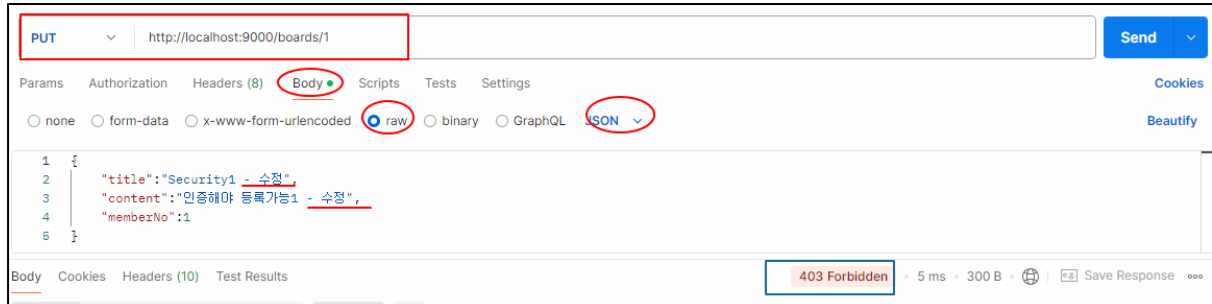




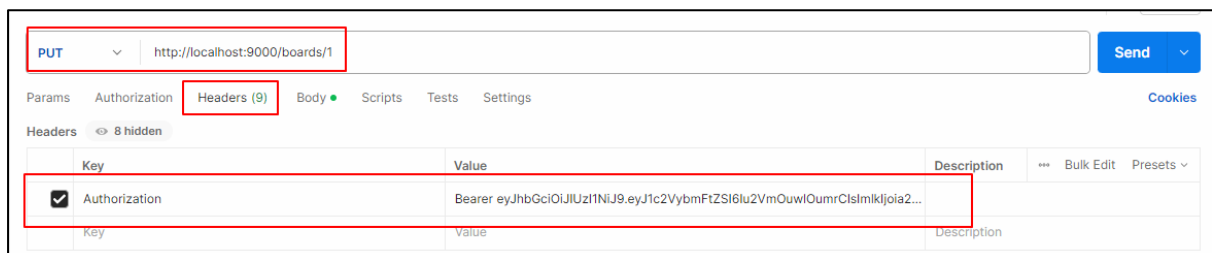
동일한 토큰값으로 계속 게시물 작성 가능하다
이때 서버를 멈춘 후 다시 가동하더라도 인증은 유지된다.

게시글수정-put 방식 <http://localhost:9000/boards/{id}> - authenticated()

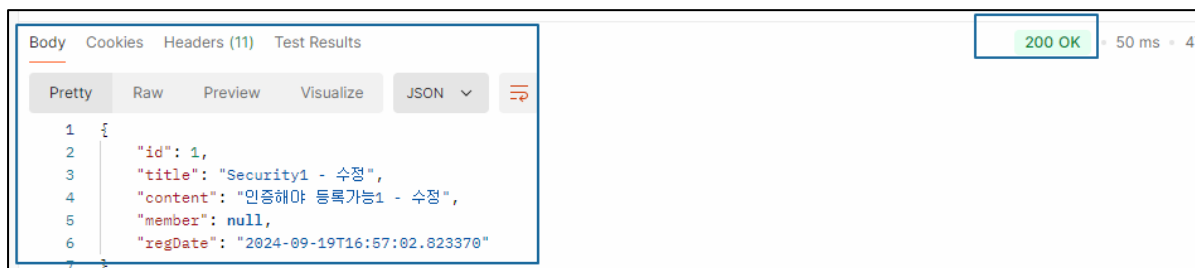
*게시글 수정하기 - 인증없이 접근해서 에러



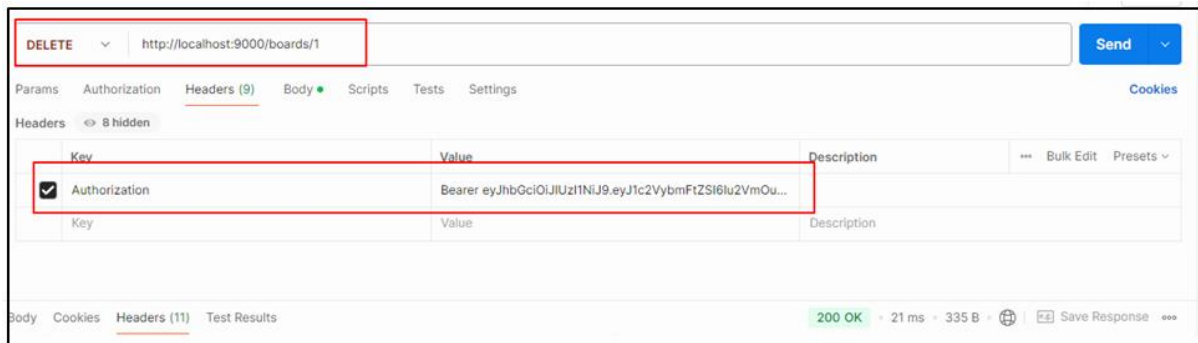
*게시글 수정하기 - 로그인 인증정보 헤더에 넘김



*게시글 수정하기 - 성공 응답

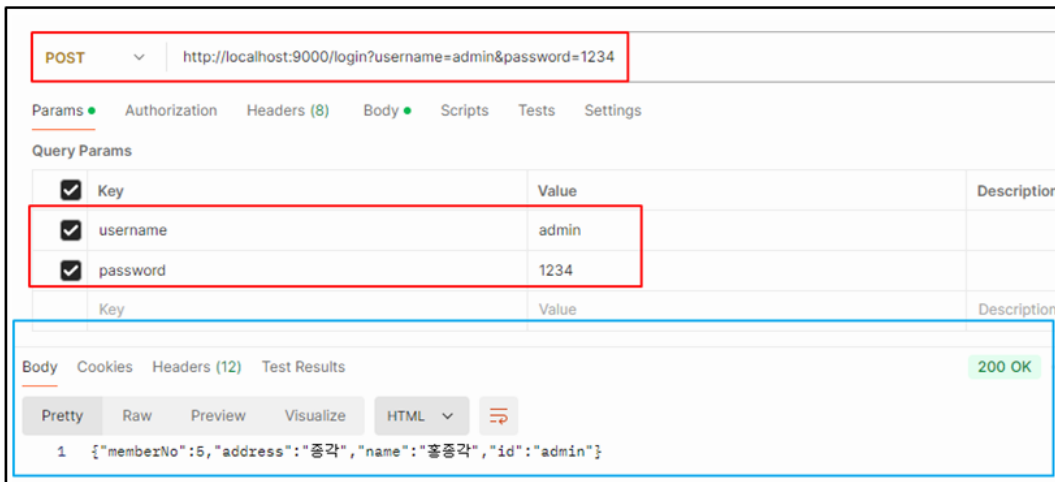


게시글삭제 - delete 방식 <http://localhost:9000/boards/{id}> - authenticated()



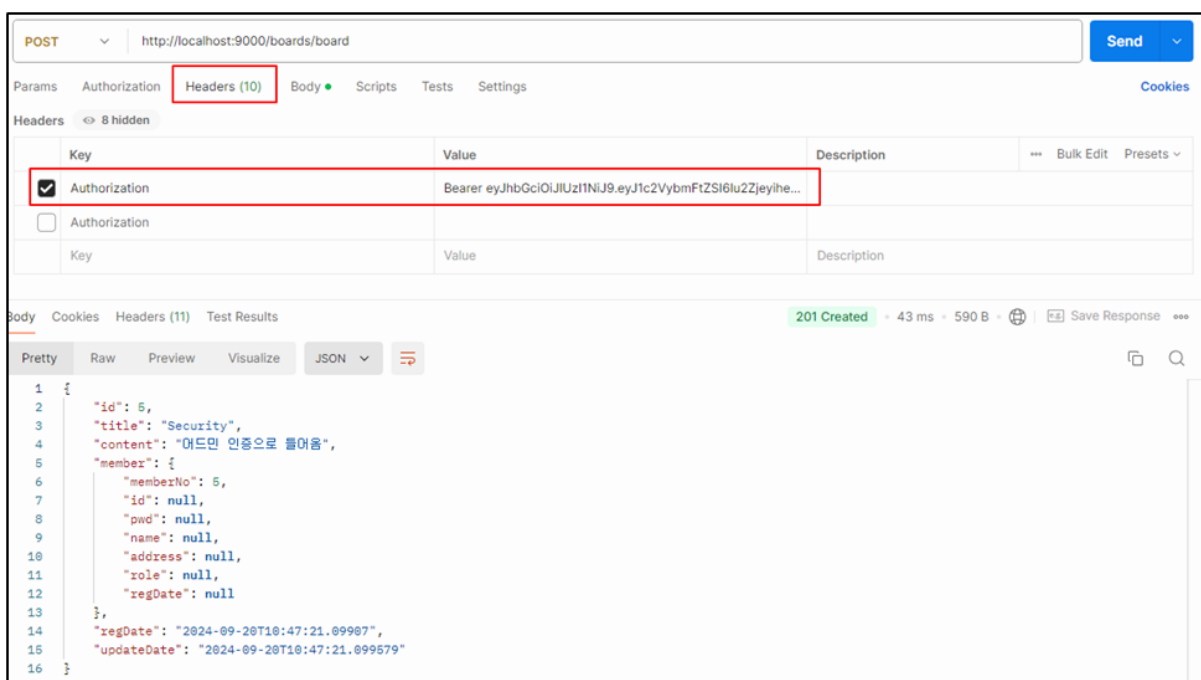
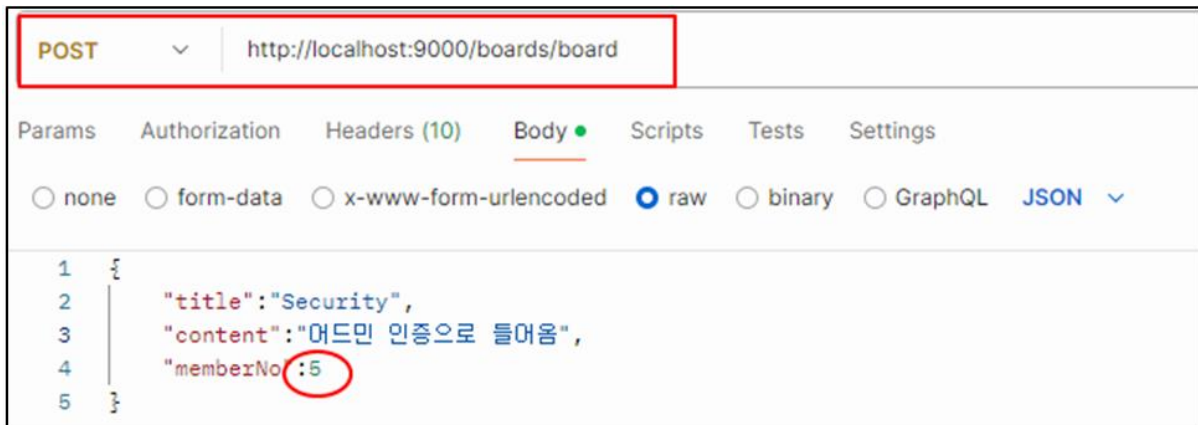
관리자- 인증 필요

*ROLE_USER role사용자로 로그인하고 접속

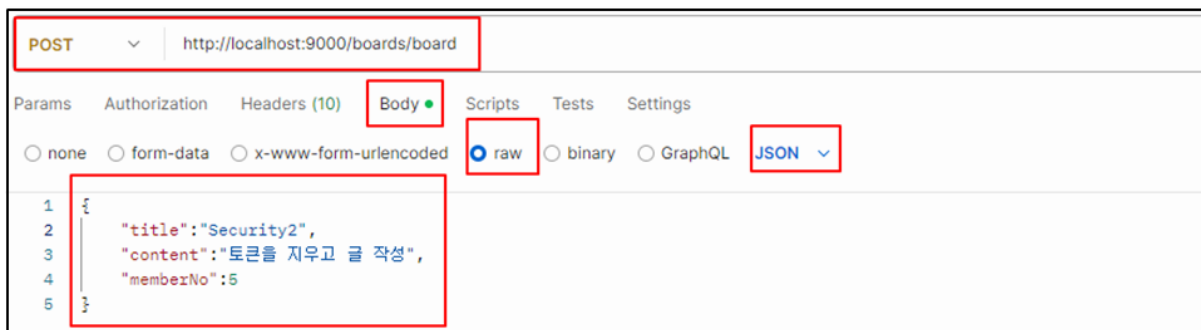


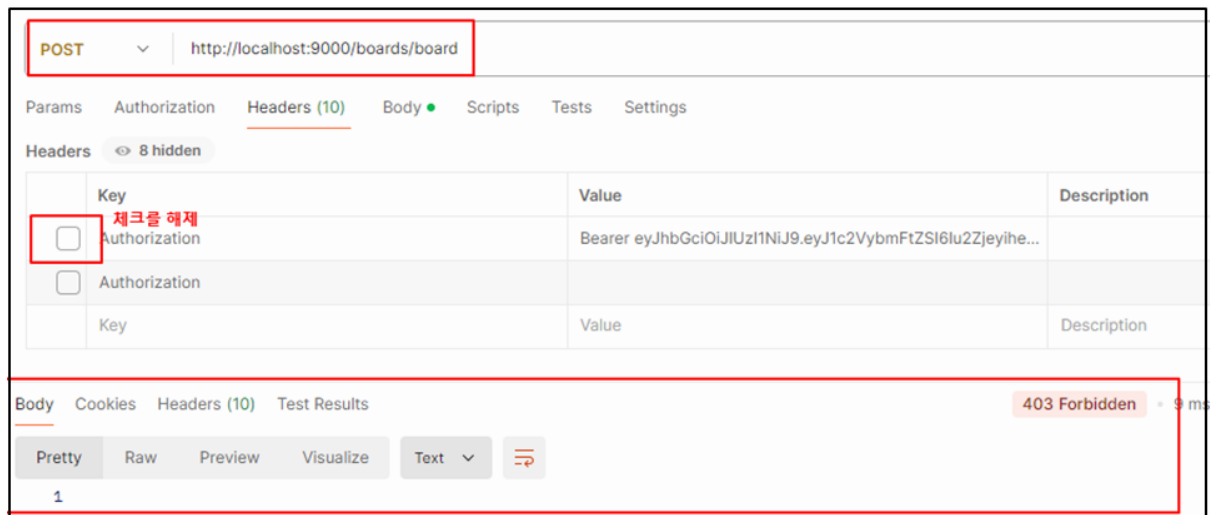
*인증된 토큰 값으로 게시 글 작성





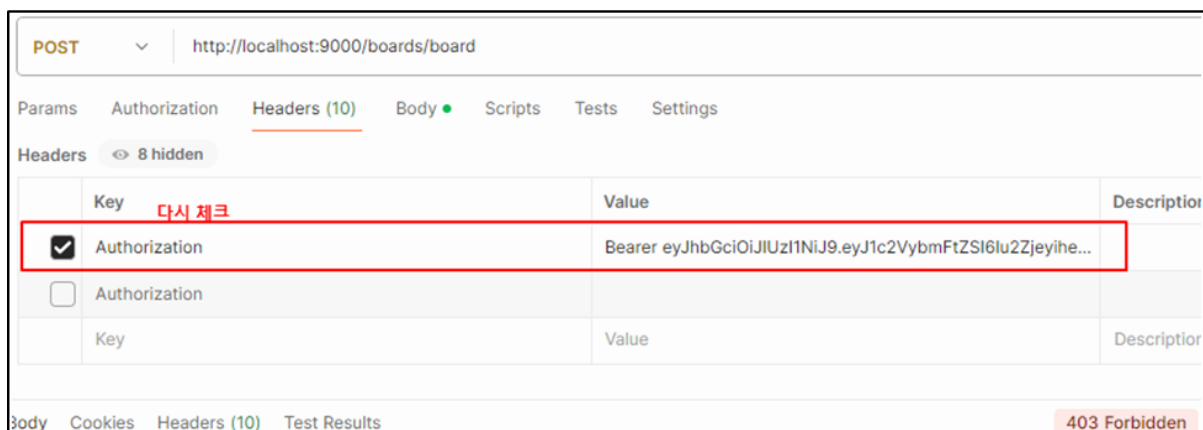
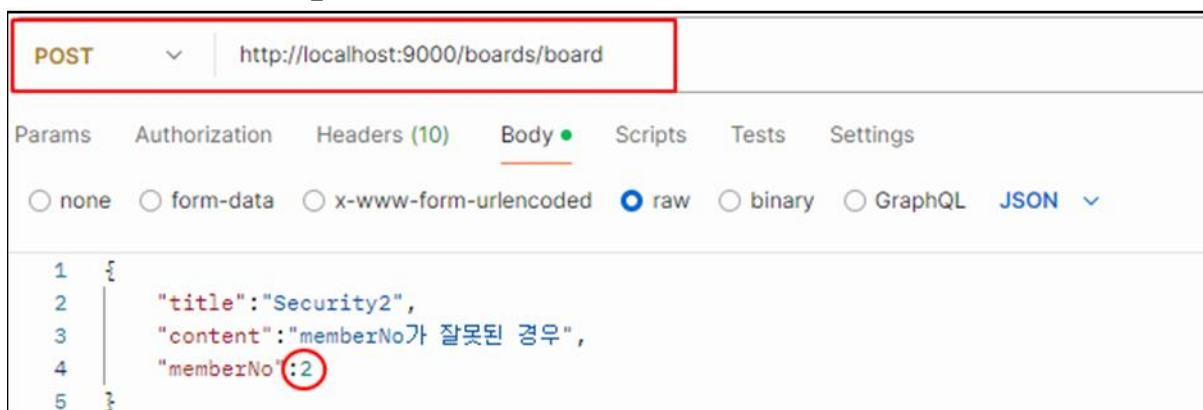
*글등록을 하나더 작성 → 이때 토큰을 안보냄 (인증 안함)





JWTFILTER에서 if조건에 걸림 → (인증 안하고 들어와서) token null 출력 → Front 단에서 처리

*인증이 있어도 member_no가 틀리는 경우



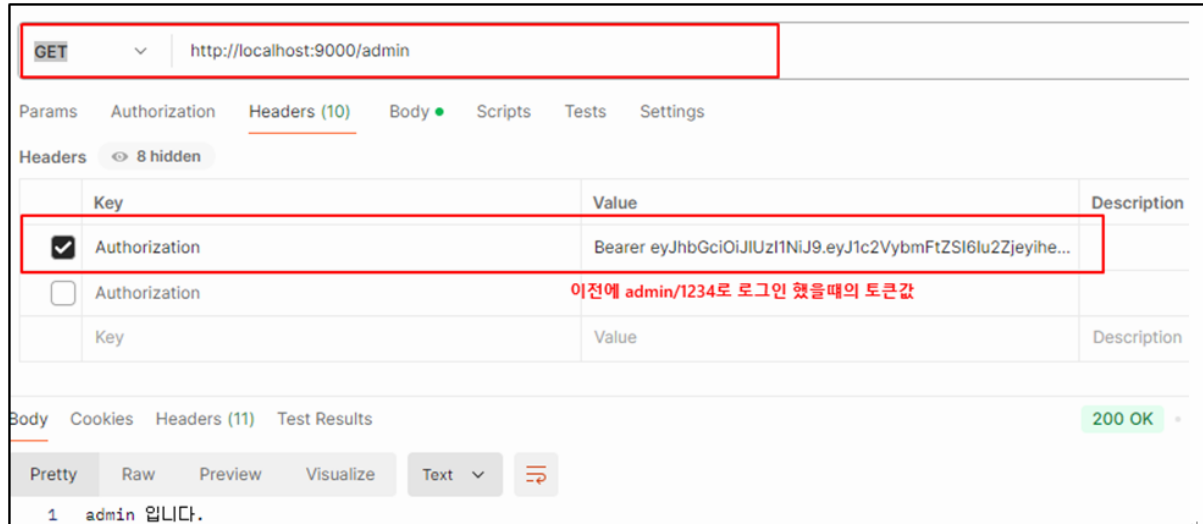
authorization now

2024-09-20T11:04:06.311+09:00 INFO 9512 -

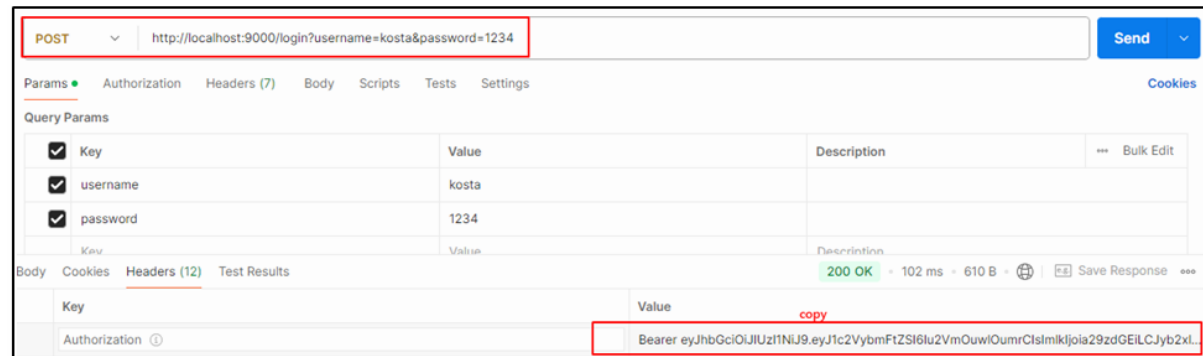
2024-09-20T11:04:06.313+09:00 ERROR 9512 -

→토큰은 있지만(인증 성공) 잘못된 아이디넘버로 에러터진다.

*admin 요청→성공 확인



*admin 요청이 | 다른 사람의 Token으로 대체(→인증은 되었지만 Role이 결여된 상황)



콘솔창 확인

authorization now

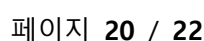
```
@GetMapping("/admin")
public String admin(){
    //시큐리티에 저장된 정보 조회
    String name = SecurityContextHolder.getContext().getAuthentication().getName();
    Log.info("Authentication getName = { } ", name);
}
```

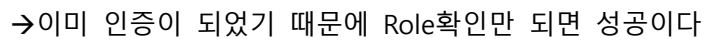
이 부분 출력안됨...

인증은 성공했지만 Role체크에서 부적합 판정이 난 경우이다.. 그래서 AdminController를 탈수 없다.

- 1) 일단 로그인 인증 과정을 거쳐서 들어오는 요청이어야 함(ID값이 admin, pwd가 1234인 값을 가져야 하는 게 중요한 게 아니라 인증 절차만 일단 거치면 된다는 뜻)
- 2) ADMIN 이라는 ROLE을 함께 가지고 있는 요청

```
@Test
void adminInsert() {
    String encPwd=passwordEncoder.encode("spring");
    memberRepository.save(Member.builder()
        .id("spring")
        .pwd(encPwd)
        .role("ROLE_ADMIN")
        .address("뉴욕")
        .name("제임스")
        .build());
}
```





SpringSecurity Architecture

The diagram illustrates the Spring Security Architecture, showing the flow of requests and the internal components.

Legend:

- Blue arrow: JWT 인증예 (JWT Authentication Example)
- Red arrow: JWT 인증예 (JWT Authentication Example)

Components and Flow:

- DelegatingFilterProxy** and **FilterChainProxy** are the entry points for the security filters.
- The **SecurityFilterChain** contains a sequence of filters: **JWTFilter**, **LoginFilter (UsernamePasswordAuthenticationFilter)**, and others.
- The **SecurityContextHolder** holds the **SecurityContext** and **Authentication** object.
- The **Controller**, **Service**, and **Repository** layers handle the business logic.
- The **AuthenticationManager** and **AuthenticationProvider** are responsible for authenticating the user.
- The **CustomMemberDetails (UserDetails)** and **CustomMemberDetailsService (UserDetailsService)** provide user details.
- The **Repository** layer interacts with the database.

페이지 21 / 22

